

Rohit Raj Singh day 7

1. Class

A **class is a blueprint/template** for creating objects.

It allows us to bundle data (variables, called *attributes*) and behavior (functions, called *methods*) together into a single, reusable structure.

Think of it like a Student form that defines what information and actions a student will have.

```
class Student:  
    pass
```

Here, `Student` is a class.

Simple meaning:

Class = Blueprint/Structure for the objects.

2. Object

An object is a real world entity created from a class. Or also defined as the instance of the class.

```
class Student:  
    pass
```

```
s1 = Student()  
s2 = Student()
```

Example:

```
Class → Student  
      ↓  
Objects → Rohit  
          Rahul  
          Aman
```

3. Constructor

A constructor is a special method that runs automatically when we create an object.

In Python, the constructor is:

```
__init__()
```

Example:

```
class Student:  
    def __init__(self, name, age):  
        self.name = name  
        self.age = age
```

When we create:

```
student1 = Student("Rohit", 22)
```

Python automatically calls:

```
__init__("Rohit", 22)
```

Simple meaning:

Constructor = Automatically runs when object is created

In Python, there are mainly **2 types of constructors** you should know:

1. Default Constructor

A constructor that does not take extra arguments except `self`.

```
class Student:
    def __init__(self):
        print("Student object created")

student1 = Student()
```

Output:

```
Student object created
```

Simple meaning:

Default constructor = **No extra data is required when creating the object.**

2. Parameterized Constructor

A constructor that takes values/parameters when creating an object.

```
class Student:
    def __init__(self, name, age):
        self.name = name
        self.age = age
```

4. Attributes

Attributes are the data/properties of an object.

Example:

```
class Student:
    def __init__(self, name, age):
        self.name = name
        self.age = age
```

Here:

```
self.name
self.age
```

are attributes.

- `self` refers to the current object.

Simple meaning:

Attributes = **Object's data/properties**

5. Methods

A method is a function inside a class.

Example:

```
class Student:
    def study(self):
        print("Student is studying")
```

Calling it:

```
student1 = Student()
student1.study()
```

Output:

```
Student is studying
```

Simple meaning:

Method = **Action/logics an object can perform**

6. Encapsulation

Encapsulation means keeping/wrapping up the data and the functions that work with that data together inside a class, while controlling direct access when needed. And controls data access.

Example:

```
class Student:
    def __init__(self, name):
        self.name = name
```

The student's data and behavior are kept inside the `Student` class.

Python also allows us to indicate internal/private data using `_` or `__`

```
class Student:
    def __init__(self):
        self.__marks = 90
```

Now `__marks` is intended to be accessed only through the class's controlled methods.

Simple meaning:

Encapsulation = **Keep data safe and control how it is accessed**

```
class Student:

    def __init__(self, name):
        self.name = name
        self.__marks = 90

    def get_marks(self):
        return self.__marks

student = Student("Rohit")

print(student.name)
print(student.get_marks())
```

7. Inheritance

Inheritance means one class can use or inherit properties and methods of another class.

Example:

```
class Person:

    def speak(self):
        print("Person")

class Student(Person):
    pass
```

Now Student inherits from Person.

```
student = Student()

student.speak()
```

Output:

Person

Simple meaning:

Inheritance = **Child class gets features from parent class**

Real-world example:

```
Person
↓
Student
↓
CollegeStudent
```

1. Single Inheritance

One parent → One child

```
class Parent:
    def show(self):
        print("Parent")
```

```
class Child(Parent):
    pass
```

```
Parent
↓
Child
```

2. Multiple Inheritance

One child → Multiple parents

```
class Father:
    def skills(self):
        print("Coding")
```

```
class Mother:
    def hobby(self):
        print("Painting")
```

```
class Child(Father, Mother):
    pass
```

```
Father  ─┐
          ↓
        Child
          ↑
Mother  ─┐
```

The child gets features from both parents.

3. Multilevel Inheritance

Parent → Child → Grandchild

```
class Grandparent:
    pass
```

```
class Parent(Grandparent):
```

```
pass
```

```
class Child(Parent):  
    pass
```

```
Grandparent  
  ↓  
Parent  
  ↓  
Child
```

4. Hierarchical Inheritance

One parent → Multiple children

```
class Animal:  
    def eat(self):  
        print("Eating")
```

```
class Dog(Animal):  
    pass
```

```
class Cat(Animal):  
    pass
```

```
Animal  
 /   \  
↓     ↓  
Dog   Cat
```

Both Dog and Cat inherit from Animal.

5. Hybrid Inheritance

Combination of two or more types of inheritance.

For example:

```
Animal  
 /   \  
Dog   Cat  
 \   /  
  ↓  ↓  
Pet
```

This combines different inheritance structures.

8. Composition

Composition means one class contains an object of another class.

Simple example:

```
class Engine:  
    def start(self):
```

```
print("Engine started")
```

```
class Car:
    def __init__(self):
        self.engine = Engine()

    def drive(self):
        self.engine.start()
        print("Car is driving")
```

Now:

```
car = Car()
car.drive()
```

Output:

```
Engine started
Car is driving
```

Simple meaning:

Composition = **One object contains/uses another object**

9. Class Methods

A class method works with the class itself, rather than a particular object.

It uses:

```
@classmethod
```

and usually takes `cls` as the first parameter.

Simple example

```
class Student:

    college = "ITS Engineering College"

    @classmethod
    def change_college(cls, new_college):
        cls.college = new_college
```

Now:

```
Student.change_college("ABC College")
print(Student.college)
```

Output:

```
ABC College
```

The method changes the class-level value.

10. Static Methods

A **static method** is a method that doesn't need object or class data.

A **static method** is simply a normal function placed inside a class because the function is related to that class.

It does **not need**:

- `self` → object
- `cls` → class

We use `@staticmethod`.

- Its Keep a function inside a class when that function is related to the class, but it doesn't need any object or class data.

It uses:

`@staticmethod`

Example:

```
class Student:

    @staticmethod
    def is_adult(age):
        return age >= 18
```

Call:

```
Student.is_adult(22)
```

Output:

```
True
```

It doesn't need `self` or `cls`.

Simple meaning:

Static method = **A useful function kept inside a class**

Why Each Class Exists

Student

The `Student` class represents an individual student in the system. It stores student-related information such as ID, name, and age, and provides methods to display student details.

Course

The `Course` class represents a course offered by a department. It maintains the list of enrolled students and the teacher assigned to the course.

Department

The `Department` class represents an academic department. It maintains a list of courses offered by that department, creating the main `Department` → `Course` relationship.

Teacher

The `Teacher` class represents a teacher in the system. It stores teacher information such as ID, name, and subject, and can be assigned to a course.

StudentManagementSystem

The `StudentManagementSystem` class manages the overall application. It handles operations such as adding, searching, updating, deleting, enrolling students, managing courses and teachers, and running the application menu.